

Human Activity Recognition (HAR) Project

Phases 1, 2 & 3 Report

Course: Supervised Learning

Project ID: 6

Name	ID
Abdelrahman Emad Mahmoud	20230211
Mohamed Essam Abdelmoneim	20230344
Abdulrahman Ahmed Kamal	20230202
Mohamed Kamal	20230349
Abdelrahman Hossam	20221095

Table of Contents

1. Project Objective
2. Phase 1: Data Engineering, Analysis & Classical ML Modeling
 - 2.1 Data Loading & Parsing
 - 2.2 Feature Engineering from Raw Signals
 - 2.3 Preprocessing & Serialization
 - 2.4 Intelligent Data Analysis (IDA)
 - 2.5 Dimensionality Reduction: PCA & LDA
 - 2.6 Supervised Learning Implementation (Classical ML)
 - 2.7 Hyperparameter Tuning & Cross-Validation
3. Phase 2: Convolutional Neural Network (CNN)
 - 3.1 CNN Architecture & Design Decisions
 - 3.2 Training & Evaluation
4. Phase 3: Advanced Deep Learning & Generative Modeling
 - 4.1 LSTM & Bidirectional LSTM
 - 4.2 LSTM with Bahdanau Attention Mechanism
 - 4.3 Transformer Encoder Architecture
 - 4.4 Generative Modeling: Conditional GAN (cGAN)
 - 4.5 Explainable AI with SHAP
5. Cross-Phase Results & Final Comparison
6. Conclusions

1. Project Objective

The goal of this project is to develop a robust supervised learning pipeline capable of classifying human physical activities — Walking, Walking_Upstairs, Walking_Downstairs, Sitting, Standing, and Laying — using tri-axial inertial signal data collected from smartphone accelerometers and gyroscopes. The project is structured in three progressive phases: Phase 1 establishes the data engineering foundation and applies classical ML models, Phase 2 introduces a Convolutional Neural Network (CNN) trained on raw signals, and Phase 3 advances to recurrent and transformer architectures alongside generative modeling.

The UCI HAR Dataset was recorded from 30 volunteers aged 19–48 performing the six activities while wearing a Samsung Galaxy S II on their waist. Signals were sampled at 50 Hz and segmented into 2.56-second windows of 128 timesteps each, yielding 10,299 labeled samples split into 7,352 training and 2,947 test samples across 9 inertial signal channels.

2. Phase 1: Data Engineering, Analysis & Classical ML Modeling

Focus: Data Acquisition, Cleaning, Exploratory Analysis, Dimensionality Reduction, and Supervised Model Benchmarking.

2.1 Data Loading & Parsing

The UCI HAR Dataset was programmatically downloaded and extracted via kagglehub. The 561 pre-computed features from X_train.txt were aligned with activity labels from y_train.txt and subject identifiers from subject_train.txt. Column names were assigned using the feature descriptors in features.txt, enabling interpretable downstream analysis.

- 561 features per sample covering time-domain and frequency-domain statistics.
- Activity labels mapped to human-readable names via activity_labels.txt.
- Raw inertial signals (9 channels × 128 timesteps) extracted separately for deep learning phases.

2.2 Feature Engineering from Raw Signals

Beyond the pre-computed features, 90 hand-crafted statistical features were extracted from the 9 raw inertial signal channels to serve as input for Phase 1 classical models:

Feature Group	Statistics Computed	Count
Time-Domain (7 per channel)	Mean, Std, Min, Max, Median, Skewness, Kurtosis	63 features
Frequency-Domain (3 per channel)	Dominant Frequency (FFT), FFT Energy, Spectral Entropy	27 features

Spectral entropy was computed by treating the normalized FFT power spectrum as a probability distribution and applying the Shannon entropy formula, yielding a measure of signal complexity that distinguishes periodic activities (Walking) from irregular or static ones (Sitting).

2.3 Preprocessing & Serialization

All 90 engineered features were standardized using StandardScaler (zero mean, unit variance) fitted exclusively on training data and applied to test data — preventing data leakage. The processed datasets were serialized to train_features.csv and test_features.csv as stable checkpoints for downstream modeling.

Data quality checks confirmed zero null values across both splits and no duplicate rows in the training set. Class distribution analysis revealed a near-balanced dataset with roughly 1,200–1,400 samples per activity class.

2.4 Intelligent Data Analysis (IDA)

Before modeling, a thorough IDA was conducted to understand the signal structure and validate preprocessing decisions:

- **Signal Patterns:** Raw body acceleration along the X-axis showed clear periodic oscillations for WALKING with a dominant frequency around 1–2 Hz, while STANDING and LAYING produced near-flat signals — confirming strong separability between dynamic and static activities.
- **Feature Correlation:** A correlation heatmap over the first 20 features revealed strong positive correlations between XYZ axes of the same sensor type (e.g., body_acc_x and body_acc_y), motivating the use of PCA for dimensionality reduction.
- **Statistical Distribution:** Histograms and KDE plots confirmed approximately normal distributions for mean and standard deviation features, satisfying the Gaussian assumptions of the Naive Bayes classifier.
- **Class Balance:** Bar chart of activity counts confirmed no significant imbalance in the training set, making standard accuracy a reliable evaluation metric.

2.5 Dimensionality Reduction: PCA & LDA

Principal Component Analysis (PCA) was applied to the 90-feature space to analyze explained variance. Results showed that approximately 20–25 components were sufficient to explain 95% of the total variance, confirming significant redundancy among correlated sensor axes. A 2D PCA projection revealed partial separation between activity clusters, with dynamic activities (Walking variants) forming distinct regions from static ones (Sitting, Standing, Laying).

Linear Discriminant Analysis (LDA) was applied as a supervised alternative, projecting the data onto 2 discriminant components that maximize class separability. The LDA visualization produced substantially cleaner cluster separation than PCA — particularly isolating LAYING from STANDING and SITTING — demonstrating that label information provides strong guidance for projection in this dataset.

2.6 Supervised Learning Implementation (Classical ML)

Four classification models were trained on the 90 standardized features and evaluated on the held-out test set using Accuracy, Weighted F1 Score, and ROC AUC (one-vs-rest):

Model	Type	Accuracy	F1 (W)	ROC AUC	Note
KNN (k=5)	Instance-Based	0.8901	0.8895	0.9872	Baseline distance similarity
Gaussian Naive Bayes	Probabilistic	0.7621	0.7598	0.9612	Gaussian distribution assumption

Random Forest (100)	Ensemble Bagging	0.9268	0.9263	0.9961	Non-linear, overfitting resistant
Gradient Boosting (100)	Ensemble Boosting	0.9401	0.9398	0.9974	Sequential error reduction

Gradient Boosting achieved the best performance across all metrics. Its sequential nature — where each tree corrects residual errors of the previous — made it particularly effective at handling the subtle boundaries between similar activities like SITTING and STANDING.

2.7 Hyperparameter Tuning & Cross-Validation

GridSearchCV with 3-fold cross-validation was applied to KNN and Random Forest to identify optimal hyperparameters:

Model	Search Space	Best Params	Validation Strategy
KNN	n_neighbors: [3, 5, 7, 9, 11]	k=5	3-fold CV
Random Forest	n_estimators: [50, 100], max_depth: [None, 10, 20]	100 trees, depth=None	3-fold CV

Cross-validation confirmed that the best parameters generalized well — the CV accuracy closely matched test accuracy, indicating no overfitting from the tuning process.

3. Phase 2: Convolutional Neural Network (CNN)

Focus: Temporal Pattern Capturing using raw inertial signals with a 1D CNN architecture.

Phase 2 transitions from hand-crafted features to raw inertial signals (shape: samples × 128 timesteps × 9 channels), allowing the CNN to learn its own temporal representations directly from sensor data.

3.1 CNN Architecture & Design Decisions

A three-block 1D CNN was designed to extract local temporal patterns from the raw signals. Conv1D layers act as learned filters that scan across the time dimension, detecting recurring motifs such as the periodic peak pattern of a walking stride.

Layer	Output Shape	Description
Input	(128, 9)	Raw signals — 128 timesteps × 9 channels
Conv1D Block 1	(64, 64)	64 filters, kernel=5, BN + ReLU + MaxPool(/2)
Conv1D Block 2	(32, 128)	128 filters, kernel=3, BN + ReLU + MaxPool(/2)
Conv1D Block 3	(32, 256)	256 filters, kernel=3, BN + ReLU + GlobalAvgPool
Dense Head	(64,)	Dense(128) + Drop(0.4) → Dense(64) + Drop(0.3)
Output	(6,)	Softmax — 6 activity classes

Batch Normalization after each Conv1D layer normalizes activations across the batch, stabilizing training and allowing higher learning rates. MaxPooling halves the time dimension after each block, reducing computation and creating translational invariance. GlobalAveragePooling in the final block aggregates all remaining timestep features into a single vector without adding trainable parameters.

3.2 Training & Evaluation

The CNN was compiled with Adam (lr=0.001) and categorical crossentropy loss. EarlyStopping (patience=10) and ReduceLROnPlateau (factor=0.5, patience=5) prevented overfitting. Training ran for up to 60 epochs with a 15% validation split and batch size of 64.

Final test accuracy: 91.55% with 169,542 parameters — the best performing deep learning model across all phases.

4. Phase 3: Advanced Deep Learning & Generative Modeling

Focus: Recurrent architectures, attention mechanisms, transformer-based models, generative data synthesis, and model explainability.

4.1 LSTM & Bidirectional LSTM

Long Short-Term Memory networks process the 128-timestep sequence step-by-step, maintaining a hidden state that acts as memory. Three gating mechanisms control information flow:

- **Forget Gate:** Decides what fraction of the previous hidden state to discard based on the current input and prior state.
- **Input Gate:** Controls how much of the newly computed candidate state is written to memory.
- **Output Gate:** Determines the portion of the current cell state exposed as the hidden state output.

Model	Architecture	Parameters	Accuracy
LSTM	LSTM(128, return_seq=True) → Drop(0.3) → LSTM(64) → Dense(64)	124,614	90.50%
BiLSTM	BiLSTM(128, return_seq=True) → Drop(0.3) → BiLSTM(64) → Dense(64)	314,310	89.41%

The two-layer stacked design was chosen because the first LSTM learns low-level temporal patterns (e.g., rate of acceleration change), while the second layer learns sequences of those patterns (e.g., the cadence rhythm of a walk).

The Bidirectional LSTM wraps each LSTM layer, running one instance forward (timestep 0→127) and another backward (127→0), then concatenating their outputs. Despite higher accuracy potential, BiLSTM used 314K parameters — the most of any model — and achieved slightly lower test accuracy (89.41%) than vanilla LSTM, suggesting some overfitting on this dataset size.

4.2 LSTM with Bahdanau Attention Mechanism

Standard LSTM compresses the entire 128-timestep sequence into a single fixed-size vector at the final timestep, which can lose information from early timesteps. The Bahdanau Attention mechanism addresses this by computing a weighted average over all 128 LSTM output states, where the weights indicate the importance of each timestep.

Attention Computation — the mechanism operates in three steps:

- **Scoring:** $\text{score}(t) = V \cdot \tanh(W \cdot h(t))$ — a learned projection of each hidden state $h(t)$ into a scalar relevance score.
- **Normalization:** $\alpha(t) = \text{softmax}(\text{score}(t))$ — converts scores to a probability distribution over timesteps that sums to 1.

- Context Vector: $c = \sum(\alpha(t) \cdot h(t))$ — weighted average of all hidden states, emphasizing the most relevant timesteps.

The context vector c replaces the final hidden state as input to the classification head. Final test accuracy: 89.35% with only 87,558 parameters — the most parameter-efficient deep learning model in Phase 3.

Attention Visualization: An attention extractor sub-model was built to expose the attention weights. Per-activity heatmaps showed which of the 128 timesteps received the highest attention weight for each class. Dynamic activities (WALKING, WALKING_UPSTAIRS) showed sharp, concentrated attention peaks at periodic intervals corresponding to stride events, while static activities (LAYING, SITTING) showed more diffuse, uniform attention distributions.

4.3 Transformer Encoder Architecture

The Transformer Encoder replaces recurrence entirely with Multi-Head Self-Attention, processing all 128 timesteps simultaneously in parallel. This is fundamentally different from LSTM — instead of reading the sequence step-by-step, the Transformer computes relationships between every pair of timesteps in a single operation.

Input Projection & Positional Encoding: A Dense layer projects each timestep from 9 → 64 dimensions (embed_dim). A learned positional encoding is added to each timestep's vector — an Embedding layer maps each position index (0–127) to a unique 64-dimensional vector that is summed with the projected features.

Two identical encoder blocks are stacked. Each block contains:

- Multi-Head Self-Attention (4 heads): $\text{attention}(Q, K, V) = \text{softmax}(QKT / \sqrt{d}) \cdot V$. With 4 heads (key_dim=16 each), the model learns 4 different types of inter-timestep relationships in parallel.
- Feed Forward Network: Dense(128, ReLU) → Dense(64) applied independently to each timestep.
- Residual Connections + LayerNorm: Each sub-layer output is added back to its input before LayerNormalization. This prevents vanishing gradients.

Stage	Shape	Operation
Input	(128, 9)	Raw inertial signals
Input Projection	(128, 64)	Dense: 9 → 64 dimensions per timestep
+ Positional Encoding	(128, 64)	Adds learned position vectors
Transformer Block 1	(128, 64)	MHA (4 heads) + FFN + Residual + LayerNorm
Transformer Block 2	(128, 64)	Deeper temporal relationship learning
GlobalAvgPool1D	(64,)	Average all 128 timestep vectors
Dense(64) + Drop(0.3)	(64,)	Task-specific classification features
Softmax Output	(6,)	6 class probability distribution

The Transformer achieved 90.53% accuracy with only 72,134 parameters — the smallest model in Phase 3. This demonstrates that the parallelized self-attention mechanism is highly parameter-efficient for temporal classification tasks.

4.4 Generative Modeling: Conditional GAN (cGAN)

To address potential class imbalance in training data and explore synthetic signal generation, a Conditional Generative Adversarial Network was implemented. The cGAN extends the original GAN framework by conditioning both generator and discriminator on class labels, enabling targeted synthesis of signals for specific activity classes.

Generator Architecture:

Component	Shape	Description
Noise Input	(100,)	Random Gaussian noise
Label Embedding	(50,)	Learned class embedding, Embedding(6, 50)
Concatenate	(150,)	Noise + label embedding
Dense + Reshape	(8, 128)	Initial temporal seed
Conv1DTranspose ×3	(128, 9)	Upsample 8→16→32→128, tanh output

Discriminator: Receives a signal (128, 9) and a label embedding concatenated along the channel axis to form (128, 18). Three Conv1D layers with LeakyReLU(0.2) activation process the combined tensor, followed by GlobalAveragePooling and Dense(1, sigmoid) real/fake output.

Training Strategy: Adam with lr=0.0002 and beta_1=0.5 was used for both networks. Training ran for 300 epochs on the four minority classes (SITTING, STANDING, WALKING_UPSTAIRS, WALKING_DOWNSTAIRS), generating 200 synthetic samples per class. Qualitative evaluation confirmed that the cGAN successfully learned the statistical properties of each activity: periodic oscillations in Walking variants and smooth near-constant patterns for static activities.

4.5 Explainable AI with SHAP

SHAP (SHapley Additive exPlanations) was applied to the best Phase 1 tree model to provide interpretable explanations of individual predictions. SHAP values are grounded in cooperative game theory — each feature's contribution is computed as its average marginal contribution across all possible feature orderings.

Explainer Selection: Gradient Boosting was identified as the best Phase 1 model. Since shap.TreeExplainer does not support multiclass GradientBoostingClassifier, shap.KernelExplainer was used instead with a background sample of 100 training points. For Random Forest, shap.TreeExplainer was applied directly on the full test set.

Sensor Channel	Mean SHAP	Physical Meaning
Total Acc Y	0.0288	Most important — lateral acceleration
Total Acc X	0.0256	Global body acceleration along X-axis
Body Gyro X	0.0086	Rotational rate around X-axis
Body Acc X	0.0080	Gravity-subtracted linear acceleration

Total Acceleration channels (X and Y) dominated feature importance, confirming that the net device acceleration is the strongest discriminator between activity classes. Per-class SHAP breakdown revealed different dominant features for different activities: dynamic activities relied on acceleration magnitude and dominant frequency features, while static activities relied more on low-variance features like kurtosis and signal range.

5. Cross-Phase Results & Final Comparison

Phase 1 — Classical Machine Learning:

Model	Input Features	Accuracy	F1 (W)	ROC AUC	Note
Gradient Boosting	Engineered (90)	0.9401	0.9398	0.9974	Best classical model
Random Forest	Engineered (90)	0.9268	0.9263	0.9961	Strong ensemble
KNN (k=5)	Engineered (90)	0.8901	0.8895	0.9872	Distance baseline
Gaussian NB	Engineered (90)	0.7621	0.7598	0.9612	Weakest — Gaussian assumption

Phase 2 — CNN on Raw Signals:

Model	Input	Accuracy	F1 (W)	Parameters	Note
CNN (1D)	Raw (128×9)	0.9155	0.9154	169,542	Best deep learning model

Phase 3 — Recurrent & Transformer Models on Raw Signals:

Model	Input	Accuracy	F1 (W)	Parameters	Note
LSTM	Raw (128×9)	0.9050	0.9050	124,614	Solid sequential baseline
BiLSTM	Raw (128×9)	0.8941	0.8938	314,310	Highest parameter count
LSTM + Attention	Raw (128×9)	0.8935	0.8941	87,558	Best interpretability
Transformer Encoder	Raw (128×9)	0.9053	0.9041	72,134	Most parameter-efficient

Key observation: Gradient Boosting on hand-crafted features (94.01%) outperforms all deep learning models on raw signals (best: CNN at 91.55%). This suggests that for this dataset size (~7,000 training samples), the domain-specific feature engineering in Phase 1 provides stronger signal than what the deep models can learn autonomously from raw data. However, deep learning models require no domain knowledge for feature design and scale better with larger datasets.

Model Efficiency Analysis:

Model	Parameters	Accuracy	Acc per 1K Params	Efficiency
CNN	169,542	91.55%	0.0054%	High — local filters

Transformer Encoder	72,134	90.53%	0.0126%	Highest — global attention
LSTM + Attention	87,558	89.35%	0.0102%	High — selective focus
LSTM	124,614	90.50%	0.0073%	Moderate
BiLSTM	314,310	89.41%	0.0028%	Lowest — over-parameterized

7. Conclusions

7.1 Key Conclusions

- Feature engineering remains competitive: Gradient Boosting on 90 hand-crafted features (94.01%) outperformed all deep learning models on raw signals, demonstrating that domain knowledge is still valuable for moderate-scale HAR datasets.
- CNN is the best deep learning architecture for this task: Its local filter design efficiently captures the periodic and transient patterns in 1D sensor signals, achieving 91.55% accuracy with 169K parameters.
- Transformers are highly parameter-efficient: The Transformer Encoder reached 90.53% with only 72K parameters — 57% fewer than CNN — making it the preferred architecture for deployment in memory-constrained environments.
- Attention mechanisms provide interpretability: LSTM + Attention not only classifies signals but also identifies which timesteps are most critical per activity, offering explainability that pure black-box models lack.
- Generative modeling enables data augmentation: The cGAN successfully synthesized realistic sensor signals conditioned on activity class, providing a principled method for addressing class imbalance without simple oversampling.
- SHAP confirms sensor importance: Total acceleration (X and Y axes) dominates the prediction signal, validating the physical intuition that net device motion is the primary differentiator between activity classes.